



JADAI

Del problema a un producto verificable

Disección del problema · Ideación · SDD · Desarrollo con IA

**La IA
acelera
lo que el
equipo
ya decidió.**

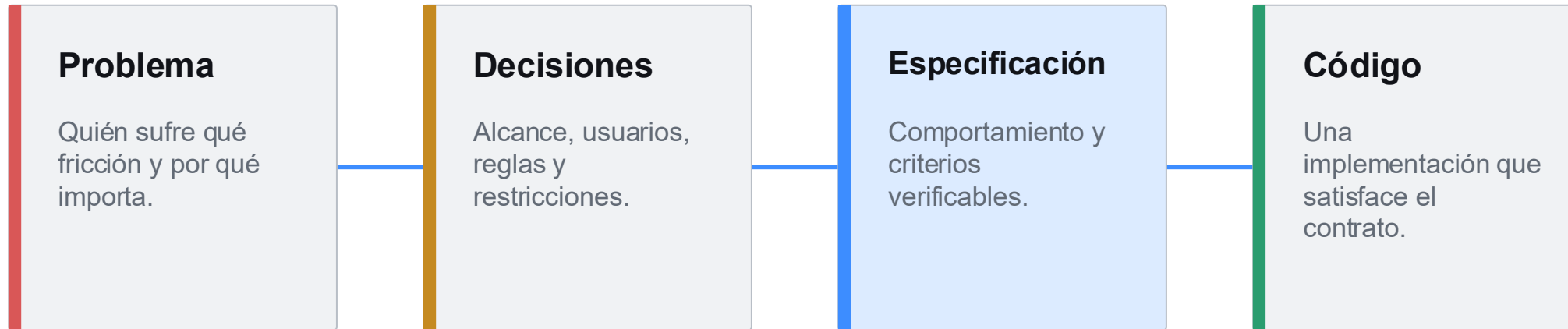
La IA no corrige una idea confusa

La ejecuta más rápido.

VELOCIDAD × AMBIGÜEDAD

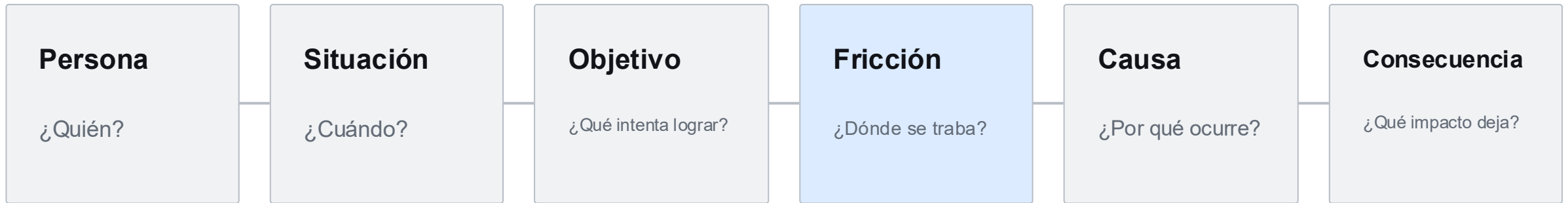
= retrabajo más rápido

El código es una respuesta, no el punto de partida



Si una decisión no está escrita, el agente tendrá que inventarla.

Disecccionar el problema revela dónde intervenir



La solución todavía no aparece en esta cadena.

Una buena frase de problema cambia la conversación

DÉBIL

“Necesitamos una app con IA para organizar voluntarios.”

Ya contiene una solución y no explica la fricción.

ÚTIL

El coordinador necesita reasignar turnos durante un evento porque las ausencias llegan por varios chats y la lista queda desactualizada.

Expone persona, situación, causa y consecuencia.

Primero abrimos opciones; después elegimos

DIVERGIR

- Una idea por línea
- Incluir una opción sin IA
- Incluir una solución de proceso
- No evaluar mientras se genera



CONVERGER

- Impacto
- Evidencia
- Factibilidad
- Diferenciación
- Riesgo

La idea ganadora maximiza aprendizaje, no funciones



El MVP debe probar la hipótesis más riesgosa.

Actividad: redacten el problema antes de diseñar

USUARIO

SITUACIÓN

OBJETIVO

FRICCIÓN

CAUSA

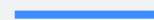
CONSECUENCIA

Todavía no escriban la solución.

La especificación es la fuente de verdad.

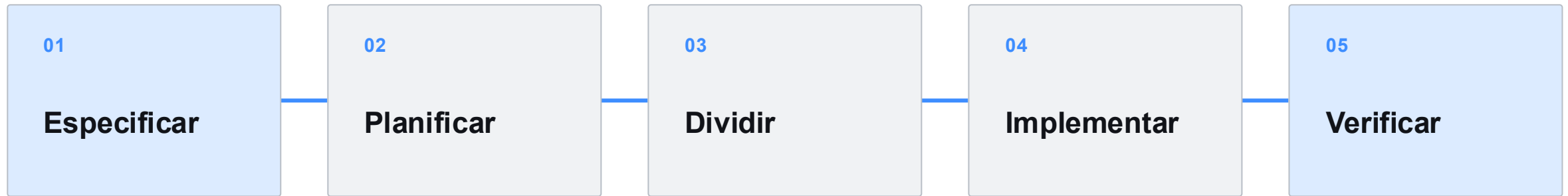
El código es una salida regenerable que debe satisfacerla.

SPEC



CODE

SDD convierte intención en un ciclo verificable



Cuando cambia el producto, cambia primero la intención.

Una spec útil reduce ocho tipos de ambigüedad

01

Objetivo

Resultado que debe cambiar

02

Alcance

Incluye y excluye

03

Usuarios

Roles y permisos

04

Workflows

Camino feliz y alternos

05

Datos

Entidades y reglas

06

Arquitectura

Stack y restricciones

07

Experiencia

Pantallas y estados

08

Aceptación

Evidencia de terminado

Cada sprint carga solo el contexto que necesita

SPRINT 01

sprints/sprint-01/

- sprint.md
- context.md

context.md

- docs/03-requisitos.md
- docs/04-arquitectura.md
- src/auth/
- tests/auth/

01

INICIAR

En el chat: “Inicia Sprint 1”

02

INYECTAR

El sprint carga las rutas de context.md

03

CONFIRMAR

La IA resume objetivo, restricciones y aceptación

04

EJECUTAR

Trabaja solo con el alcance y archivos relevantes

Menos contexto no significa menos información: significa información relevante para el sprint actual.

El ecosistema open source cubre método y ejecución

Herramienta	Enfoque	Cuándo usarla
GitHub Spec Kit	Spec → Plan → Tasks → Implement	Estructurar SDD con cualquier agente.
OpenSpec	Specs y deltas junto al código	Mantener contexto por capacidad.
BMAD Method	Brainstorming, producto y arquitectura	Guiar descubrimiento con roles.
OpenHands	Agent harness para repositorios	Ejecutar agentes con herramientas y estado.

Fuentes: github.github.io/spec-kit · openspec.dev · github.com/bmad-code-org · github.com/OpenHands

Un modelo responde; un harness mantiene el trabajo

MODELO

Prompt
↓
Respuesta

No conserva por sí solo el estado del proyecto.

HARNESS

**Lee archivos · usa terminal
ejecuta pruebas · guarda estado
divide tareas · evalúa resultados
repite hasta cumplir**

Convierte una conversación en un proceso de ingeniería.

El método funciona con cualquier agente de programación

1

Crear repo

Un espacio Git limpio

2

Escribir contexto

Decisiones en Markdown

3

Consolidar spec

app_spec.txt

4

Dividir tareas

Incrementos pequeños

5

Implementar

Una tarea por vez

6

Revisar

Diff + pruebas + aceptación

La aceptación convierte opiniones en evidencia

DADO

que un coordinador tiene un turno sin cubrir

CUANDO

publica una solicitud de reemplazo

ENTONCES

la primera aceptación válida asigna el turno

Y

las respuestas posteriores ven “ya cubierto”

Debe poder demostrarse, probarse o inspeccionarse.

Antes de construir, el equipo debe poder responder esto



¿Quién tiene el problema?



¿Qué consecuencia queremos cambiar?



¿Qué queda fuera del MVP?



¿Qué decisiones no puede inventar la IA?



¿Cómo sabremos que cada flujo funciona?



¿Qué integración necesita un mock o fallback?

CIERRE

La IA amplifica la claridad del equipo.

Problema claro. Decisiones explícitas. Criterios verificables.

**Pensar antes
de construir
no reduce la
velocidad.**

**Evita correr
en la
dirección
equivocada.**